COL 1101

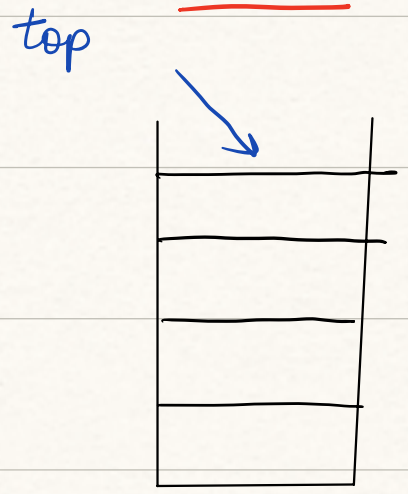
LECTURE 11

AUGUST 14, 2026

STACKS & QUEUE APPLICATIONS

Two ADTs

STACK

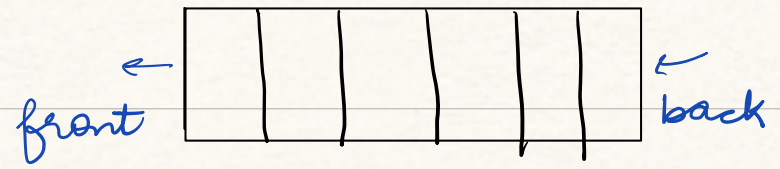


Last-in-First-Out

(LIFO)

operations	ArrayStack	LinkedStack
pop	O(1)*	O(1)
push(x)	O(1)*	O(1)

QUEUE



First-in-First-Out

(FIFO)

operations	Array	Linked
enqueue(x)	O(1)*	O(1)
dequeue	O(1)*	O(1)

Today's class: Applications of stacks and queues

BALANCED PARANTHESES PROBLEM

Problem: Given a string containing brackets of three kinds
( ), [ ], { }. Decide whether they are properly matched

- critical for syntax validation (and more) in compilers, editors etc.

( )

[ ]

{ }

BALANCED PARANTHESES PROBLEM

Problem: Given a string containing brackets of three kinds ( ), [ ], { }. Decide whether they are properly matched

- every opening bracket has a matching closing bracket in the correct order

BALANCED PARANTHESES PROBLEM

Problem: Given a string containing brackets of three kinds ( ), [ ], { }. Decide whether they are properly matched

- every opening bracket has a matching closing bracket in the correct order

Example: 1) [( )] { } → Y

2). ([ )] → N

3). (((} } → N

How to solve it?

BALANCED PARANTHESES PROBLEM

Problem: Given a string containing brackets of three kinds ( ), [ ], { }. Decide whether they are properly matched

- every opening bracket has a matching closing brack in the correct order

Solution: count the # of brackets of each kind

(( )) ✓

([ )] → NOT balanced

BALANCED PARANTHESES PROBLEM

Problem: Given a string containing brackets of three kinds ( ), [ ], { }. Decide whether they are properly matched

- every opening bracket has a matching closing brack in the correct order

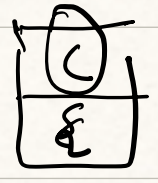
Solution: Most recently opened should be first to be closed. Last IN, first out

BALANCED PARANTHESES PROBLEM

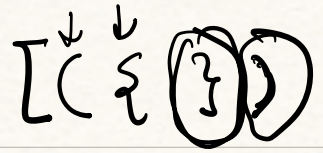
Most recently opened should be the first to be closed.

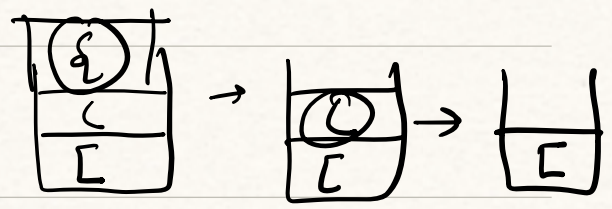
Algorithm :

```
balanced (s)
    stack = ArrayStack()
    for c in s:
        if c is opening bracket:
            stack.push(c)
        elif c is closing bracket:
            if stack.is_empty():
                return False
            elif stack.pop() does not match c:
                return False
    ↳ if not stack.isempty() : return False
    return True
```



BALANCED PARANTHESES PROBLEM





```python
def balanced(s):
    match = {")": "(", "]": "[", "}": "{"}
    stack = ArrayStack()
    for ch in s:
        if ch in "([{":
            stack.push(ch)                          # remember an opener
        elif ch in ")]}":
            if stack.is_empty() or stack.pop() != match[ch]:
                return False                        # nothing to close, or wrong type
    return stack.is_empty()                         # leftover openers -> unbalanced
```

TIME COMPLEXITY : O(n)

FINDING SHORTEST PATH IN A MAZE

Given a maze, start (S), and exit (E), find the shortest way out.

Constraints : 1) walls

2). can move up/left/down/right, one step at a time

(0,0)

2D list



Find all path from S to E and return the shortest path.

↓

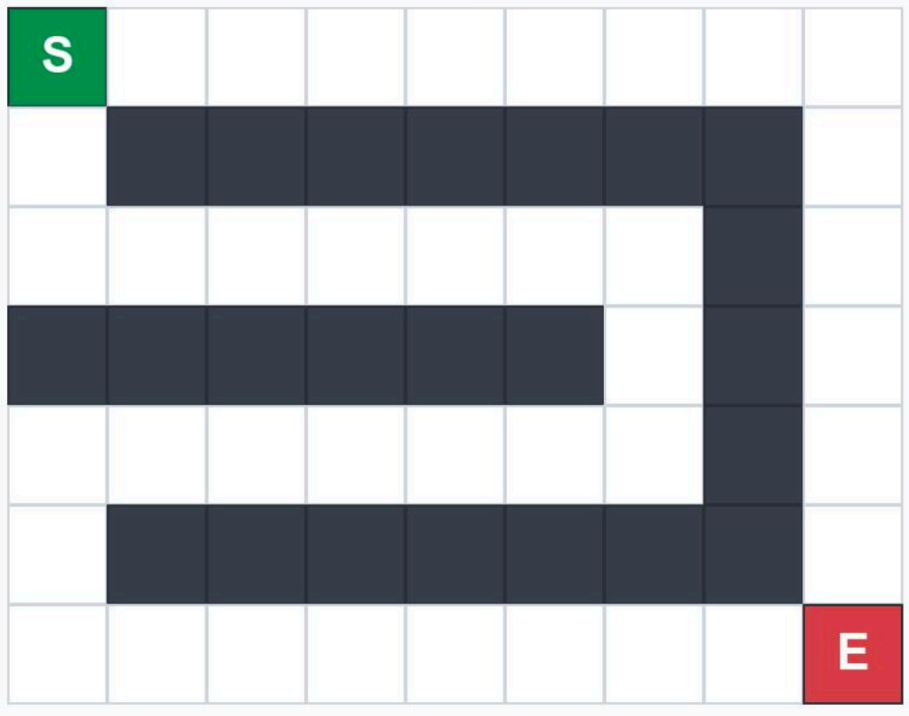
(n-1, n-1)

exponential ← (very expensive)

FINDING SHORTEST PATH IN A MAZE

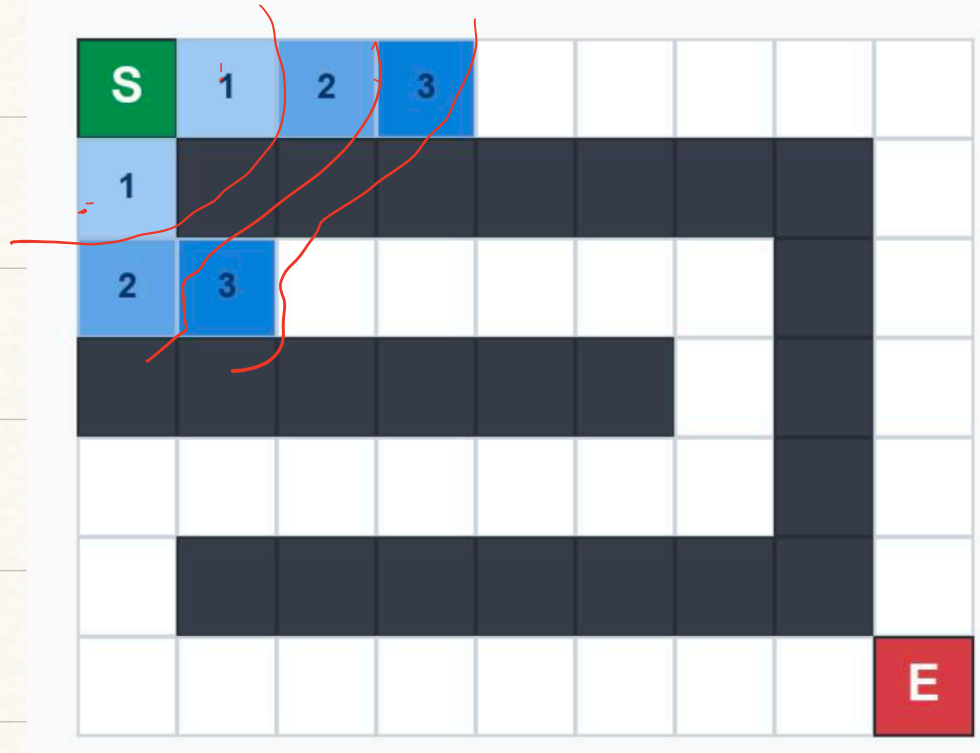
Goal : Find, shortest path by construction

.



IDEA: EXPLORE IN LAYERS

- Start from S
- Explore everything 1-step away
- From those, everything 2-step away



IDEA: EXPLORE IN LAYERS

- Start from S
- Explore everything 1-step away
- From those, everything 2-step away
- and continue



IDEA: EXPLORE IN LAYERS

- Start from S
- Explore everything 1-step away
- From those, everything 2-step away
- Stop when E is reached
- Return path by backtracking

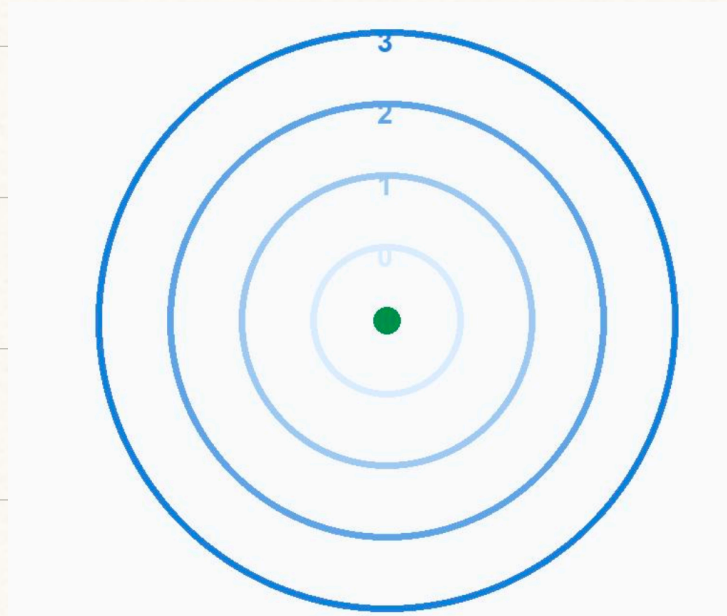
Shortest path by construction

[Not a formal proof!]

Proof by



WHICH DATA STRUCTURE?



- Create an empty queue Q
- Enqueue (0,0) and set its dist = 0
- while Q is not empty
  - dequeue its first element c
  - enqueue all unexplored neighbors b of c
  - dist (b) = dist (c) + 1
  - set parent to c
  - stop if b == E

ALGORITHM IN ACTION



Dictionaries [dist[[0,0]] = 0
parent = ϕ
None

Q: [0,0],

step 2 : ([0,1], [1,0])

step 3 : ([1,0], [0,2])

step 4 ([0,2], [2,0])

dist [(0,1)] = dist [(0,0)] + 1 = 1

dist [(1,0)]

dist

Breadth-First Search

CODE

```python
from collections import deque

def bfs(grid, start, goal):
    R, C = len(grid), len(grid[0])
    def is_open(r, c):
        return 0 <= r < R and 0 <= c < C and grid[r][c] != '#'
    dist = {start: 0}
    prev = {start: None}                    # for rebuilding the path
    q = deque([start])
    while q:
        r, c = q.popleft()                  # oldest frontier cell -- the FIFO
        if (r, c) == goal: break
        for dr, dc in ((1,0), (-1,0), (0,1), (0,-1)):
            nr, nc = r + dr, c + dc
            if is_open(nr, nc) and (nr, nc) not in dist:
                dist[(nr, nc)] = dist[(r, c)] + 1
                prev[(nr, nc)] = (r, c)
                q.append((nr, nc))          # enqueue the new ring
    return dist, prev
```

FINDING SHORTEST PATH

```
def shortest_path(prev, goal):
    path, cur = [], goal
    while cur is not None:          # hop backwards: goal -> ... -> start
        path.append(cur)
        cur = prev[cur]
    return path[::-1]               # reverse to read start -> goal
```

APPLICATION # 3 : EVALUATING EXPRESSIONS

Calculate $3 + 4 * 2$ = 11

↓

infix notation

$3 + 8 = 11$

APPLICATION #3 : EVALUATING EXPRESSIONS

Calculate $3 + 4 * 2$

Infix notation : operator sits b/w operands

Problem: Need to know precedence to evaluate

APPLICATION #3 : EVALUATING EXPRESSIONS

Calculate $3 + 4 * 2$

Infix notation : operator sits b/p operands

Problem: Need to know precedence to evaluate

↳ can't evaluate in a single left-to-right scan

APPLICATION #3 : EVALUATING EXPRESSIONS

Calculate $3 + 4 * 2$

Infix notation : operator sits b/p operands

Problem : Need to know precedence to evaluate

↳ can't evaluate in a single left-to-right scan

Reverse Polish Notation (RPN)

Postfix : Operator is written after its two operands

E.g. $3\ (4\ 2\ *)\ +$

$3\ 8\ +$

$= 11$

APPLICATION #3 : EVALUATING EXPRESSIONS

Calculate $3 + 4 * 2$

Infix notation : operator sits b/p operands

Problem: Need to know precedence to evaluate

↳ can't evaluate in a single left-to-right scan

Postfix : Operator is written after its two operands

No parantheses, precedence rules needed!

Infix	Postfix
$3 + 4 * 2$	$3\ 4\ 2\ *\ +$
$(3+4) * 2$	$3\ 4\ +\ 2\ *$

Evaluating Postfix Expressions

Write a function to evaluate postfix expressions.

E.g. $3\ (4\ 2)\ *\ +$

queue or stack?

$3\ 8\ +$ → LIFO

```
postfix(e)
    stack = ArrayStack()
    for token in e:
        if is_operand(token):
            stack.push(token)
        if is_operator(token):
            b = stack.pop()
            a = stack.pop()
```

a + b

stack.push(c)

return stack.pop()

operate (a, b, tok

Evaluating Postfix Expressions

Write a function to evaluate postfix expressions.

```
import operator
def eval_postfix(tokens):
    ops = {"+": operator.add, "-": operator.sub,
           "*": operator.mul, "/": operator.truediv}
    stack = ArrayStack()
    for t in tokens:
        if t in ops:
            b = stack.pop()          # top = second operand
            a = stack.pop()          # first operand
            stack.push(ops[t](a, b))
        else:
            stack.push(float(t))     # an operand -- stash it
    return stack.pop()
```

MAXIMUM IN EVERY WINDOW

Problem: A stream of readings and a window of last K, sliding forward one step at a time. At every step, report the maximum inside the window.

MAXIMUM IN EVERY WINDOW

Problem: A stream of reading and a window of last K, sliding forward one step at a time. At every step, report the maximum inside the window.

Example: input [[4 [2 [5] [1] 6] 3]] K = 3

output ([5 5 6 6])

Brute force: Iterate over each window

m = find max (cur. window)

out.append(m)

return out

n, K

O(nK)

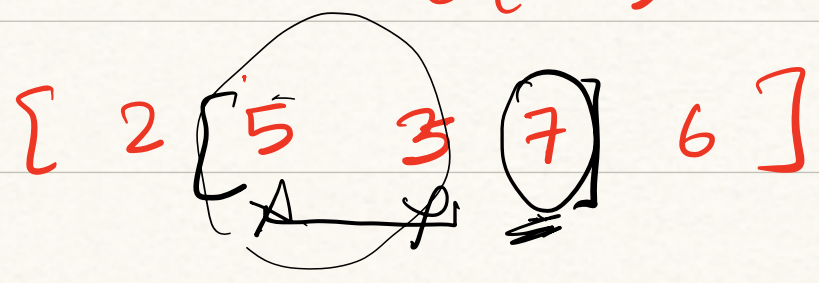
can we do better?

MAXIMUM IN EVERY WINDOW

Problem: A stream of reading and a window of last K, sliding forward one step at a time. At every step, report the maximum inside the window.

Approach #1: For each window, scan its values and take max.

$O(nk)$



MAXIMUM IN EVERY WINDOW

Key idea: When a bigger value arrives, the smaller values before are useless.

[[4 2 5] 1 6 3]

[4 2 5]

Keep a sequence of decreasing elements

[5]

MAXIMUM IN EVERY WINDOW

Key idea: When a bigger value arrives, the smaller values before are useless.

Invariant: Store indices whose values run strictly decreasing front → back.

CODE

```python
from collections import deque

def max_in_windows(a, k):
    dq = deque()
    out = []
    for i, x in enumerate(a):
        if dq and dq[0] <= i - k:
            dq.popleft()
        while dq and a[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            out.append(a[dq[0]])
    return out
```